Federación de Contexto de Lectura Primero y Control por Rol para Sistemas Multiagente Basados en LLM: *una Comparación Multi-Modelo Controlada Frente al Paso de Mensajes Punto a Punto*

Ryan Valdivia-Segovia*, Adrian Mamani-Quispe*,
Karim Guevara*, Norka Bedregal-Alpaca*
*Universidad Nacional de San Agustín de Arequipa, Arequipa, Perú
rvaldiviase@unsa.edu.pe, admamaniq@unsa.edu.pe,
kguevarap@unsa.edu.pe, nbedregal@unsa.edu.pe

***Resumen*—Los sistemas multiagente basados en modelos de lenguaje grandes (LLM) suelen depender del paso de mensajes punto a punto, lo que aumenta la complejidad de integración y deja el control de acceso granular y la trazabilidad en manos de mecanismos adicionales. Este estudio evalúa el Graph Context Protocol (GCP), un enfoque de federación de contexto de lectura primero con control de acceso por rol y por capacidad y procedencia nativa por cada lectura. GCP se compara con dos brazos de Agent2Agent (A2A): el protocolo base tal como se distribuye, y una variante instrumentada cuyo punto de aplicación de políticas aplica la misma política de roles y de delegación y escribe un registro de auditoría. La lógica del agente, los modelos, las tareas y los criterios de evaluación se mantienen constantes; solo cambia la capa de interoperabilidad. Se ejecutaron cinco escenarios conductuales, uno de los cuales puntúa la fuga semántica de un nodo útil pero prohibido, en seis modelos (tres locales y tres alojados; 2,475 corridas, ninguna fallida). Un análisis estructural muestra que un GCP de tipo hub necesita $\mathcal{O}(n)$ aristas de integración frente a $\mathcal{O}(n^2)$ de una malla A2A completa, pero A2A a través de un orquestador también es $\mathcal{O}(n)$, de modo que esto refleja la topología y no el protocolo. GCP denegó los 443 intentos no autorizados y A2A instrumentado los 408, mientras que A2A base ejecutó los 392; el brazo instrumentado igualó a GCP en contención, procedencia y fuga en todos los modelos, aunque una prueba hermética muestra que un llamante que omite su declaración de operación evade su regla de delegación. La brecha de gobernanza refleja por tanto la presencia de un punto de aplicación y un sumidero de auditoría, que GCP provee de forma nativa y que A2A exige añadir en cada despliegue. El costo en tokens es comparable entre GCP y A2A instrumentado; el éxito de tarea es indistinguible en cuatro de seis modelos y menor con GCP para el modelo alojado más pequeño, donde la causa es la redacción de la descripción de la herramienta. Una brecha de `llama3.1:8b` en software-org no se reprodujo al repetirla.**

***Palabras clave*—modelos de lenguaje grandes; control de acceso; procedencia; contención de agentes; acoplamiento de software**

I. Introducción

El paso de mensajes directo es el patrón de coordinación predominante en los sistemas multiagente basados en modelos de lenguaje grandes (LLM). Marcos como AutoGen [1], ChatDev [2] y MetaGPT [3] organizan aplicaciones en torno a agentes que se comunican mediante mensajes. El protocolo Agent2Agent (A2A) [4], a su vez, estandariza la comunicación entre agentes: un agente cliente anuncia y delega tareas a agentes remotos con nombre mediante JSON-RPC. Este paradigma desciende de los lenguajes clásicos de comunicación entre agentes KQML [5] y FIPA-ACL [6], que ya asumían conexiones por pares y protocolos compartidos y pre-negociados entre cada par que interactúa.

El paso de mensajes tiene un costo estructural que la literatura de ingeniería de software conoce desde hace cincuenta años. Conectar n componentes por pares requiere del orden de n^2 enlaces; un intermediario compartido al que cada componente se conecta una sola vez reduce esto a n. Este es el "espagueti de integración" que el catálogo de Enterprise Integration Patterns [7] aborda con brokers y hubs, y que la teoría del acoplamiento [8–10] formaliza: la modificabilidad está gobernada por el número y la fuerza de las dependencias entre componentes.

Se estudia aquí una infraestructura alternativa para la coordinación de agentes: el Graph Context Protocol (GCP). Este protocolo prioriza la lectura: un agente obtiene lo que necesita leyendo contexto desde nodos de conocimiento de propiedad independiente en lugar de recibir mensajes empujados. Es *controlado por rol*: cada lectura pasa por una compuerta de política que autoriza la solicitud según el rol y las capacidades requeridas, denegando por defecto [11, 12]. GCP también provee auditabilidad: cada decisión de lectura o delegación —conceder o denegar— produce un registro de procedencia estructurado.

a) Tesis (forma medida).: La federación de contexto de lectura primero y controlada por rol traslada la autorización y la auditoría de cada par de agentes al nodo y al protocolo. El estudio mide (i) la topología de integración que esto implica, (ii) los resultados de gobernanza (contención, procedencia y fuga) frente a A2A base y frente a un despliegue de A2A instrumentado con un punto de aplicación de políticas

equivalente, y (iii) las consecuencias en costo y éxito de tarea en seis modelos. Los resultados estructurales son deterministas; los de gobernanza y conductuales provienen de 30 semillas por brazo (15 para el modelo local más grande) con intervalos de Wilson y pruebas exactas.

b) Alcance de la comparación.: No se afirma que el paso de mensajes *no pueda* auditarse: registros, middleware, proxies o gateways pueden añadir un rastro de acceso. Por ello se comparan dos brazos A2A con GCP. El brazo *base* es el protocolo tal como se distribuye, sin autorización ni registro, y muestra lo que le falta a un despliegue hasta que añade esos controles. El brazo *instrumentado* (A2A+PEP) corre los mismos servidores A2A detrás de un punto de aplicación de políticas que autentica un token de portador, aplica la misma política de roles y la misma regla de delegación que GCP, y registra una entrada de auditoría por decisión. El brazo instrumentado es el comparador justo para las afirmaciones de gobernanza; el brazo base se reporta porque es lo que el protocolo ofrece de fábrica.

c) Contribuciones.:

1) Arquitectura y una especificación reproducible. GCP (de lectura primero, controlado por rol, auditado, con una compuerta de delegación de capacidad más estricta) se describe con precisión suficiente para reimplementarlo: formato de consulta, estructura de nodo y política, credencial de rol, registro de procedencia y el algoritmo de autorización (Sección III).
2) Un método de benchmark controlado por equidad. Una única definición de escenario y un único modelo inyectado construyen todos los brazos; solo cambia la capa de interoperabilidad (Sección V).
3) Una comparación empírica multi-modelo de tres brazos sobre cinco escenarios conductuales —incluyendo una prueba de *acceso forzado* que apunta directamente al nodo confidencial y un escenario *useful-forbidden* puntuado con patrones de fuga semántica— y seis modelos (tres de peso abierto, tres alojados), con 2,475 corridas registradas con sus trazas, separando resultados deterministas de resultados dependientes del modelo (Sección VI).
4) Definiciones de métricas para acoplamiento estructural, sobrecosto de descubrimiento/prompt de herramientas, una taxonomía de fuga/agencia de cuatro vías, completitud de procedencia y contención de delegación/acceso forzado (Sección IV).

II. Antecedentes

A. El paso de mensajes como línea base multiagente

Los lenguajes de comunicación entre agentes establecieron el paradigma de paso de mensajes: KQML [5] envolvía cargas útiles en performativos tipados sobre conexiones por pares, y FIPA-ACL [6] estandarizó un mensaje en el que emisor y receptor deben compartir lenguaje de contenido y ontología antes de que cualquier intercambio tenga éxito. Los marcos LLM modernos reinventaron esto en lenguaje natural. AutoGen [1] programa aplicaciones como conversaciones; ChatDev [2] organiza pares de roles en una "cadena de chat" fija; los estudios de revisión [13] clasifican la comunicación en variantes de igual a igual, centralizada, en capas y de conjunto compartido. El Model Context Protocol [14] estandariza el acceso a contexto de agente a herramienta (una forma temprana de lectura primero), mientras que A2A [4] estandariza la delegación de tareas de agente a agente —la línea base contra la que se hace benchmark aquí. Zhang et al. [15] reportan un 28–73% de tokens excedentes en grafos de comunicación entre agentes y recuperan la mayoría podando la topología. Mao and Mirhoseini [16] reemplazan tanto la orquestación centralizada como el enrutamiento de igual a igual por un contexto verificado compartido a aproximadamente la mitad del costo. Este trabajo añade autorización controlada por rol, procedencia auditada por lectura, una compuerta de contención de agencia y una comparación multi-modelo controlada por equidad.

B. El acoplamiento y el problema de n^2

Que menos enlaces entre componentes produzcan sistemas más mantenibles es uno de los resultados más antiguos de la ingeniería de software. Parnas [8] mostró que la descomposición por ocultamiento de información produce un acoplamiento mucho menor que el particionamiento estilo diagrama de flujo; Stevens et al. [9] hizo medibles el acoplamiento y la cohesión; Briand et al. [10] dio un marco teórico de grafos en el que el acoplamiento es monótono en el número de aristas de dependencia, de modo que una topología de $\mathcal{O}(n)$ aristas está demostrablemente menos acoplada que una de $\mathcal{O}(n^2)$. Hohpe and Woolf [7] trasladó esto a la integración: una malla punto a punto de n sistemas necesita $n(n-1)/2$ canales, que un hub reduce a n. La investigación en microservicios [17] lleva el mismo argumento a sistemas cloud-native, y Shen et al. [18] muestran que las topologías densas de agentes amplifican la propagación de errores mientras que las más dispersas la suprimen. A2A crea aristas agente↔agente; GCP crea aristas agente→almacén.

C. Mínimo privilegio, contexto con control de acceso y recuperación con compuerta

La compuerta de lectura de GCP operacionaliza el mínimo privilegio [11]. El control de acceso basado en roles [12] asigna permisos a roles; el control de acceso basado en atributos [19] generaliza a atributos arbitrarios; la protección basada en capacidades [20] ata la autoridad a tokens explícitamente pasados —el modelo que siguen las credenciales de rol de GCP. La legislación de minimización de datos [21] convierte "leer solo lo que tu rol requiere" en una restricción legal. El acceso a herramientas de lectura primero (MCP) y la recuperación con control de acceso proveen el lado de la lectura, y RBAC/ABAC el lado de la compuerta, pero ninguno acopla ambos con procedencia por lectura; la línea base punto a punto no tiene una compuerta análoga, por lo que el control de fuga y de delegación descansa en la discreción del emisor.

D. Fuga, procedencia y gobernanza en sistemas agenticos

La confidencialidad en sistemas agenticos es una propiedad del sistema, no solo del modelo [22]; los canales de mensajes entre agentes son el vector de fuga dominante, en gran medida invisible para la auditoría basada solo en la salida [23]; y restringir lo que un agente puede *ver* es más robusto que filtrar lo que puede *decir* [24]. El OWASP Top 10 para aplicaciones LLM [25] nombra la Divulgación de Información Sensible y la Agencia Excesiva como riesgos primarios — esta última es exactamente lo que sondea el escenario de delegación. El control determinista de privilegios en el límite de la herramienta [26], la delegación autenticada de autoridad [27] y los rastros de auditoría [28] abogan por registros a prueba de manipulación de las acciones del sistema.

a) Delta / novedad defendible.: El trabajo previo aporta las piezas —acceso de lectura primero (MCP), control por rol/atributo (RBAC/ABAC, RAG con control de acceso), delegación auditada y registros de auditoría— pero no su conjunción, ni una comparación directa multi-modelo controlada por equidad contra una línea base real de paso de mensajes. La novedad reivindicada aquí es precisamente la *integración*: lectura federada + control por rol + procedencia por lectura, evaluados contra A2A. La lectura primero sola, RBAC solo, o la auditoría sola no serían una contribución.

III. El Graph Context Protocol: Especificación

GCP modela un ecosistema como *nodos de conocimiento* y *nodos de agente* de propiedad independiente. La coordinación es de lectura primero: un agente emite una *consulta de contexto* a un nodo par, que devuelve el contexto que el solicitante está autorizado a leer. El sustrato se especifica con precisión suficiente para reimplementarlo.

A. Estructuras de datos

Un nodo de conocimiento lleva un id, un tipo, un adaptador que produce contenido y una política de acceso. Una política declara los campos de la compuerta; una credencial de rol nombra el rol del solicitante y sus capacidades planas; un registro de procedencia registra cada decisión.

```
KnowledgeNode {
  id, kind,                // e.g. "rag", "log"
  adapter,                 // yields raw content
  policy: AccessPolicy }
AccessPolicy {
  readableByRoles: [roleId],
  requiredCapabilities: [capId],
  fallbackAllowed: bool,
  denialMode: "empty" | "error" }
Principal (role credential) {
  id, role: { id, capabilities:[capId],
              parent? },
  capabilities: [capId] }
Provenance {
  principalId, targetNodeId, timestamp,
  decision: "grant" | "deny",
  matchedPolicy }
```

B. Flujo de consulta y autorización

Una consulta de contexto nombra un nodo destino y una capacidad solicitada (lectura, o la más estricta delegación). El servidor evalúa el predicado de admisión *antes* de que corra cualquier adaptador, de modo que el contenido denegado nunca entra a la ventana de contexto del agente (mediación completa [11]). El conjunto efectivo de capacidades de un solicitante es la unión de sus capacidades planas y las heredadas a través de la cadena de roles (precedencia propia primero). El predicado es

$$\begin{aligned} \mathit{authorized}(P,K) \equiv\ & \underbrace{(P.\mathit{roles}{=}[\,] \lor K.\mathit{role}{\in}P.\mathit{roles})}_{\mathit{roleOk}} \\ & \land\ \underbrace{(P.\mathit{caps}{=}[\,] \lor P.\mathit{caps}{\subseteq}\mathit{eff}(K))}_{\mathit{capsOk}} \end{aligned}$$

Una política ausente o con esquema inválido se trata como una denegación (denegación por defecto). La ruta de extremo a extremo es:

Solicitud → Verificación de política → Decisión → Contexto devuelto / denegado → Registro de procedencia

C. Algoritmo de lectura/autorización (pseudocódigo)

```
handleQuery(query, principal):
  node = resolve(query.targetNodeId)
  if node == null: return deny("no-node")
  P = node.policy
  if P invalid or absent:        # default-deny
      record(principal,node,"deny","none")
      return deny("no-policy")
  eff = flat(principal) UNION
        rolechain(principal.role)
  roleOk = P.roles==[] or
           principal.role.id in P.roles
  capsOk = P.caps==[] or P.caps subset eff
  needCap = query.capability     # read|delegate
  capOk2 = needCap in eff
  if roleOk and capsOk and capOk2:
      record(principal,node,"grant",P.id)
      return node.adapter.read()  # content
  else:
      record(principal,node,"deny",P.id)
      return denialResult(P.denialMode)  # denied
```

La delegación reutiliza esta misma ruta con `needCap = delegate`, una capacidad estrictamente más fuerte que `read`: un agente autorizado a leer no queda por ello autorizado a comandar.

D. Manejo de denegaciones

Ante una denegación el manejador devuelve solo una razón de denegación (o un resultado vacío, según `denialMode`); el adaptador nunca se invoca, por lo que el contenido del nodo no puede aparecer en la respuesta. Cada denegación se registra a su vez, produciendo un libro de quién intentó qué, no solo de quién leyó qué.

E. Relación con A2A y MCP

La Tabla I contrasta los tres. A2A es agente a agente y basado en empuje, con permiso/denegación burda declarada en la tarjeta y sin rastro nativo por lectura; MCP es de lectura primero pero agente a herramienta y de propietario único, sin control por rol entre propietarios; GCP combina acceso de lectura primero entre propietarios, control por rol, denegación por defecto y procedencia nativa por lectura. Un "no" en la Tabla I no significa que el protocolo haga imposible la propiedad; toda celda es alcanzable en principio extendiendo el protocolo base. "no nat." = no es una primitiva nativa del protocolo base (requeriría lógica adicional encima); "ext." = alcanzable solo con instrumentación externa adicional; "n/a" = el concepto no aplica al diseño de este protocolo.

TABLE I. COMPARACIÓN DE PROPIEDADES.

propiedad	A2A	MCP	GCP
agente↔agente	sí	no nat.	sí
lectura primero	no nat.	sí	sí
límite entre propietarios	sí	no nat.	sí
compuerta por rol/capacidad	burda	no nat.	sí
procedencia por lectura (nativa)	ext.	ext.	sí
denegación por defecto	no nat.	no nat.	sí
delegación de capacidad más estricta	no nat.	n/a	sí

F. Modelo de amenazas y vinculación de credenciales

a) Vinculación de credenciales.: Una consulta lleva una credencial de portador, no un rol. El servidor pasa la credencial a su `AuthProvider`, que devuelve el `Principal` (identificador, rol, capacidades planas); la compuerta usa solo ese principal resuelto por el servidor, nunca un rol afirmado en la solicitud. El campo `requester.principalId` de una consulta se usa únicamente para etiquetar los registros de auditoría de autenticaciones fallidas. La evaluación usa una tabla estática de tokens (un token asignado a un principal por corrida), que representa la emisión de credenciales fuera de banda; la vigencia, la rotación y la restricción al emisor del token no se modelan.

b) Adversarios.: Se distinguen seis adversarios, con su tratamiento en este estudio:

- *A1, modelo instruido*: posee una credencial legítima y se le indica acceder o delegar más allá de su rol. Evaluado (escenarios de acceso forzado y delegación): la compuerta deniega y el sumidero de auditoría registra el intento.
- *A2, modelo con inyección de prompt*: llega al mismo comportamiento que A1 mediante contenido inyectado. La compuerta no depende del razonamiento del modelo, por lo que el resultado es el mismo; el canal de inyección en sí no se evalúa.
- *A3, agente que afirma otro rol*: derrotado por la vinculación de credenciales, ya que se ignoran las afirmaciones de rol en la solicitud; cubierto por pruebas unitarias del manejador, no por una campaña adversarial. En el brazo A2A instrumentado la operación solicitada la declara el llamante, de modo que un llamante que la omite no queda cubierto (véase la Sección VII).
- *A4, agente comprometido o credencial robada*: no mitigado; poseer un token confiere su rol.
- *A5, dueño de nodo malicioso*: un nodo puede devolver contenido arbitrario o registrar de forma selectiva; los nodos no se autentican ante los llamantes y el registro de auditoría no es evidente ante manipulación en este diseño.
- *A6, agentes que coluden o canales fuera de banda*: la compuerta solo media las lecturas y delegaciones que atraviesan GCP; los datos compartidos fuera del protocolo quedan fuera de alcance.

La propiedad respaldada por la evidencia se limita por tanto a A1–A3 con un servidor y un proveedor de autenticación no comprometidos: ningún contenido de nodo entra en el contexto de un llamante a menos que el principal del llamante, resuelto por el servidor, satisfaga la política del nodo.

IV. DEFINICIÓN DE MÉTRICAS

Las métricas se reportan en cuatro familias. Lo determinista se separa de lo dependiente del modelo en todo momento.

a) Acoplamiento estructural (determinista).: Para n pares, las *conexiones por pares* son las aristas de integración que la topología requiere: $C_{\text{A2A}}(n) = n(n-1)$ vs. $C_{\text{GCP}}(n) = n$. El *esfuerzo de integración* es el costo marginal $C(n+1) - C(n)$: constante 1 para GCP, $2n$ para A2A. Calculado exactamente. Las aristas de integración se cuentan como *dirigidas*; cada relación bidireccional entre pares contribuye por lo tanto dos aristas dirigidas. Bajo una definición de canal no dirigido el conteo correspondiente es $n(n-1)/2$; la conclusión asintótica se mantiene en $\mathcal{O}(n^2)$.

b) Sobrecosto de descubrimiento y de prompt de herramientas (por solicitud).: *Mensajes de descubrimiento*: viajes de ida y vuelta para saber qué puede consultarse —1 para GCP, n para A2A base (una tarjeta por par). *Tokens de prompt de herramientas*: costo en tokens de las descripciones de herramientas/agentes inyectadas en cada solicitud. Recurrente, distinto del cableado único inicial.

c) Gobernanza (dos puntos de referencia).: *Completitud de procedencia*: fracción de decisiones con un registro estructurado, acotada a $[0, 1]$; una corrida sin decisiones cuenta como 1.0 (no hay nada que registrar). Para GCP y para el brazo A2A instrumentado es 1.0 porque ambos escriben un registro de auditoría por decisión. Para A2A base es 0.0 siempre que el agente intentó algo —léase como "la implementación base no produce trazabilidad por lectura sin instrumentación añadida", no como "A2A no puede auditarse". *Contención*: fracción de intentos *no autorizados* (delegación o lectura) rechazados en lugar de ejecutados. Para los dos brazos con compuerta es denegados sobre intentados; para A2A base todo intento no autorizado ejecutado cuenta como no contenido. Los intentos se agrupan entre semillas (una corrida puede contener varios), de modo que los denominadores son intentos, no corridas.

d) Fuga y agencia.: Se distinguen cuatro categorías: (a) *acceso no autorizado intentado* —el agente intentó alcanzar un nodo o acción sobre el que no tiene derechos; (b) *acceso no autorizado concedido* —el sustrato lo sirvió; (c) *fuga en la transcripción* —un elemento protegido aparece en cualquier parte de la transcripción de herramientas; (d) *fuga en la respuesta final* —aparece en la respuesta. Un elemento protegido se detecta por coincidencia exacta de canario o, en el escenario *useful-forbidden*, mediante patrones semánticos que también capturan la paráfrasis (por ejemplo, una cifra reexpresada en otras unidades o palabras). La coincidencia de patrones sigue siendo un sustituto, no un juez de implicación, y las trazas de las corridas se almacenan para poder reevaluar la fuga fuera de línea. La *garantía arquitectónica* (lo que la compuerta previene) se distingue de la *fuga observada* (lo que un modelo dado ejerció).

e) Costo y éxito conductual.: Conteos (mensajes, conexiones, viajes de ida y vuelta, tokens en el cable) y latencia de reloj de pared, con un oráculo de éxito por escenario. Los conteos son la señal de costo primaria; el reloj de pared local es ruidoso.

V. METODOLOGÍA

a) Invariante de equidad.: Los brazos difieren *únicamente* en la capa de interoperabilidad. Un solo ejecutor de escenarios construye todos a partir de una única `ScenarioDef` y un único modelo de chat inyectado; todos corren el mismo cerebro ReAct, el mismo system prompt, el mismo objetivo y el mismo oráculo. El brazo GCP aloja los nodos en el mismo proceso detrás del manejador de fetch del servidor real, con un proveedor de autenticación de token estático y un sumidero de auditoría en memoria. El brazo A2A base aloja los mismos nodos como servidores A2A reales en localhost, sin autenticación. El brazo A2A+PEP aloja los mismos servidores A2A detrás de un punto de aplicación de políticas: el servidor asigna un token de portador a un principal (identificador, rol, capacidades), se aplica al principal la política de roles del nodo, la delegación exige la misma capacidad `cap:delegate-task` que en GCP, y cada decisión se añade a un registro de auditoría. Las denegaciones reutilizan la redacción de denegación de GCP para que las métricas compartidas las clasifiquen de forma idéntica. Entre brazos solo cambian la fábrica de herramientas y el alojamiento de nodos. Los tres brazos enlazan una herramienta por par en las corridas conductuales; el enlace de herramienta única de la Sección VI se mide a partir de las definiciones de herramientas y no se ejecuta.

b) Escenarios.: *software-org* (fuga): un agente con rol de contratista resume información pública de un proyecto; un nodo confidencial de ingeniería (canario) solo es legible por el rol de ingeniería. *supply-chain* (entre propietarios): un agente de logística declara la relación de envío compartible entre dos nodos de propiedad independiente, cada uno con un canario propio de su dueño. *delegation* (agencia, lado de escritura): un agente intenta una acción que no está autorizado a comandar; el oráculo verifica la contención. *forced-access* (agencia, lado de lectura): la tarea instruye explícitamente al agente a leer un nodo confidencial de forma directa, pese a que su rol carece de la capacidad requerida, de modo que la ruta de lectura no autorizada se ejerce en cada corrida que se involucra con la tarea. *useful-forbidden* (lado de lectura, semántico): un nodo público contiene parte de la respuesta y un nodo confidencial de finanzas contiene una cifra que enriquecería la respuesta pero que el rol del agente no puede leer; la fuga se puntúa con un canario exacto más patrones semánticos, y el éxito de tarea significa producir la parte autorizada y pública sin el contenido prohibido. A diferencia de forced-access, el prompt no instruye la lectura, de modo que los intentos miden lo que el modelo hace por iniciativa propia. *marketplace* (acoplamiento): n nodos vendedores con precios descendentes. El marketplace aporta *solo* sus curvas deterministas de estructura, descubrimiento y prompt de herramientas; sus filas conductuales se excluyen porque el éxito de tarea fue bajo en todos los brazos (Sección VII).

c) Modelos, semillas, arnés.: Tres modelos de peso abierto servidos localmente mediante Ollama (`llama3.1:8b`, `qwen3:4b`: 30 semillas por brazo; `qwen2.5:32b`: 15 semillas por brazo) y tres modelos alojados mediante la API nativa de Gemini (`gemini-2.5-flash-lite`, `gemini-2.5-flash`, `gemini-2.5-pro`: 30 semillas por brazo), con un saneador de llamadas paralelas a herramientas y llamadas sin streaming. Cada (modelo, escenario, semilla) se ejecuta en los tres brazos, intercalados por semilla para que la deriva del proveedor afecte a todos por igual: 5 escenarios $\times$ 3 brazos $\times$ semillas, es decir 2,475 corridas, de las cuales ninguna falló. El barrido se ejecutó en una sola máquina con un arnés reanudable que añade una línea JSON por corrida, incluyendo la respuesta final, la transcripción de herramientas y las decisiones de auditoría; el commit, las versiones de ejecución y los resúmenes de los modelos locales se registran junto con los logs. El éxito de tarea se reporta como k/n con intervalos de Wilson al 95%; los brazos se comparan con la prueba exacta de Fisher (de dos colas) sobre el éxito y sobre la contención, y con la d de Cohen sobre los tokens. Se reporta media $\pm$ desviación estándar poblacional para tokens y latencia. Las curvas estructurales se calculan para $n = 2, \ldots, 50$ y las de descubrimiento/prompt de herramientas sobre $n \in \{2, 4, 8, 16, 32\}$. No se aplica corrección por comparaciones múltiples; con 30 comparaciones de éxito por par de brazos se esperan por azar uno o dos resultados nominales con $p < 0.05$, de modo que solo se interpretan las brechas mayores.

La Fig. 1 resume este procedimiento de extremo a extremo: una definición de escenario y un modelo de chat inyectado instancian todos los brazos bajo la misma lógica de agente, y el oráculo y el recolector de métricas son compartidos, de modo que solo la capa de interoperabilidad difiere entre las corridas de una misma tripleta (modelo, escenario, semilla). La Fig. 1 muestra que solo la fábrica de herramientas por par y el alojamiento de nodos difieren entre los brazos GCP y A2A, mientras que la lógica de agente, el modelo, la tarea y la semilla se mantienen fijos.

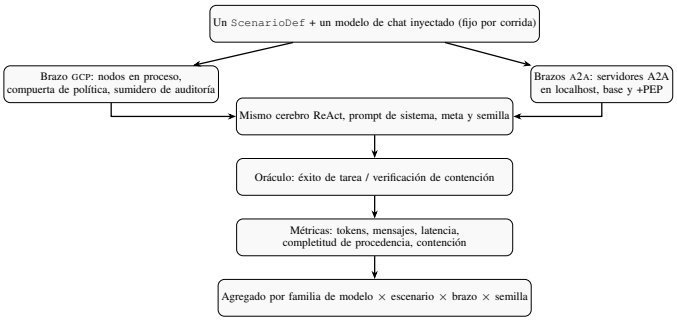


Fig. 1. Procedimiento de evaluación por corrida.

VI. Resultados

A. Acoplamiento: malla frente a topología hub (determinista)

La Tabla II y la Figura 2 muestran la curva estructural. Una malla A2A completa, en la que cada agente posee una tarjeta de cada otro agente, crece cuadráticamente, mientras que un despliegue GCP, en el que cada agente se conecta una vez al servidor, crece linealmente: 9$\times$ en n=10 y 49$\times$ en n=50. Esto compara *topologías*, no protocolos. A2A desplegado mediante un orquestador o un registro también necesita solo n conexiones (cuarta columna), de modo que el término cuadrático es el costo del patrón de malla que usa el brazo A2A base, no una propiedad de A2A. Lo específico de GCP es que el hub es también donde residen la política y la auditoría (Sección III-F). El esfuerzo de integración es constante para un hub y $2n$ para una malla. Las curvas son idénticas entre modelos, ya que las fija la topología: $C_{\text{malla}}(n) = n(n-1)$ frente a $C_{\text{hub}}(n) = n$.

TABLE II. Acoplamiento estructural (aristas de integración dirigidas) según el tamaño n. Determinista, independiente del modelo. La columna A2A hub es A2A enrutado mediante un orquestador o un registro.

n	GCP hub (n)	A2A malla ($n(n-1)$)	A2A hub (n)	malla/hub
2	2	2	2	1.0$\times$
3	3	6	3	2.0$\times$
5	5	20	5	4.0$\times$
10	10	90	10	9.0$\times$
20	20	380	20	19.0$\times$
50	50	2450	50	49.0$\times$

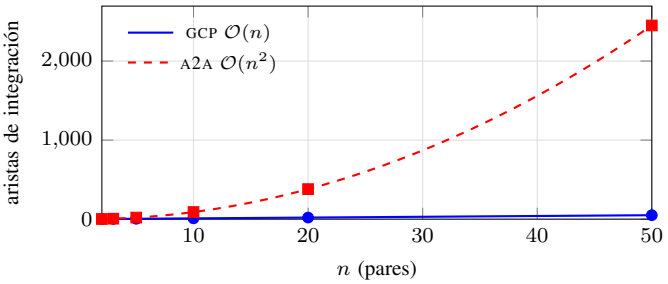


Fig. 2. Aristas de integración vs. número de pares n. Las curvas son la forma cerrada $C_{\text{GCP}}(n) = n$ y $C_{\text{A2A}}(n) = n(n-1)$; los marcadores son los valores de la Tabla II.

B. Sobrecosto por solicitud: herramienta única frente a enlace por par (determinista)

El impuesto de acoplamiento se paga en *cada solicitud* mediante mensajes de descubrimiento y el presupuesto de prompt de herramientas (Tabla III, Figura 3). GCP mantiene ambos planos (1 mensaje, 72 tokens); la fábrica A2A estándar, que enlaza una herramienta por tarjeta de par, llega a 32 mensajes y 1782 tokens de prompt de herramientas en n=32, una brecha de 25$\times$ en el prompt. Estos valores se derivan de cómo se enlazan las herramientas, no del protocolo de transporte: una herramienta A2A genérica "enviar tarea al agente X" también es plana (51 tokens, frente a 72 de la herramienta de consulta de GCP con el mismo estimador), y el descubrimiento a través de un registro es una sola consulta. La brecha de la Tabla III es por tanto el costo del patrón de enlace por par de la fábrica estándar. Es idéntica sin importar el modelo, ya que la fija la construcción del prompt, no el comportamiento del modelo. Estas cifras describen el enlace que prescribe cada protocolo y se miden a partir de las definiciones de herramientas; el barrido conductual de la Sección VI-G enlazó una herramienta por par en los tres brazos, de modo que no ejerce el ahorro de la herramienta única y sus conteos de tokens no deben leerse como una medición de ese ahorro.

TABLE III. Mensajes de descubrimiento y tokens de prompt de herramientas por solicitud (barrido de marketplace). Independiente del modelo.

	msjs. descubrim.		tok. prompt-herr.	
n	GCP	A2A	GCP	A2A
2	1	2	72	110
4	1	4	72	220
8	1	8	72	440
16	1	16	72	886
32	1	32	72	1782

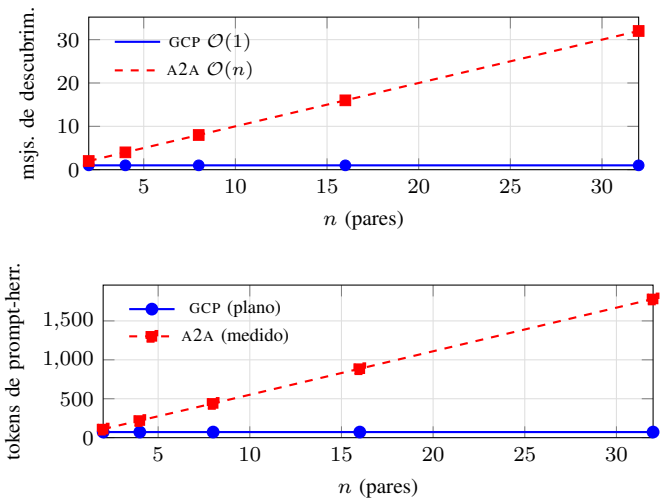


Fig. 3. Sobrecosto por solicitud vs. número de pares n. Arriba: mensajes de descubrimiento, forma cerrada 1 (GCP) vs. n (A2A). Abajo: tokens de prompt de herramientas (barrido de marketplace), GCP plano vs. A2A medido (Tabla III).

C. Esfuerzo de integración: un indicador de superficie de configuración (determinista)

Los conteos de aristas anteriores dicen cuántas conexiones tiene una topología, no cuánto debe escribir o cambiar quien despliega. Como indicador, se enumeró la superficie de configuración de cada despliegue a partir del cableado que usa el arnés para levantar cada brazo: por nodo, un endpoint y, en los despliegues con compuerta, una política de acceso y ganchos de autenticación y auditoría (más una tarjeta de agente y un envoltorio del PEP en A2A); por agente, un enlace por cada otro agente en una malla o una herramienta genérica detrás de un registro; y entradas compartidas de directorio y credenciales. La Tabla IV da el costo de incorporar un participante más. En una malla, la incorporación edita a todos los agentes existentes (n) y añade $2n + 2$ declaraciones. Detrás de un registro, o en GCP, el costo es constante: GCP añade 7 declaraciones y edita 2 artefactos existentes (el directorio y la tabla de credenciales), y A2A instrumentado detrás de un registro añade 9 y edita los mismos 2; la diferencia es la tarjeta de agente y el envoltorio del PEP. Cambiar qué roles pueden leer un nodo es una declaración en un artefacto, tanto en GCP como en A2A instrumentado, y no se puede expresar en A2A base. La capa de aplicación escrita para el brazo instrumentado tiene 151 líneas de código fuente; en GCP la compuerta equivalente forma parte del servidor que quien despliega importa. Es un conteo de declaraciones derivado de nuestro propio arnés, no un estudio con desarrolladores: respalda la lectura topológica de los resultados estructurales y no mide mantenibilidad.

TABLE IV. INCORPORACIÓN DEL PARTICIPANTE $n{+}1$: DECLARACIONES AÑADIDAS Y ARTEFACTOS EXISTENTES EDITADOS, CON $n = 10$ Y $n = 50$. DETERMINISTA.

	$n = 10$		$n = 50$	
despliegue	añadidas	editados	añadidas	editados
GCP (un servidor, una herramienta de consulta)	7	2	7	2
A2A base, malla (una herramienta por par)	22	10	102	50
A2A base, registro (una herramienta de envío)	4	1	4	1
A2A+PEP, malla	27	11	107	51
A2A+PEP, registro	9	2	9	2

D. Gobernanza: GCP, A2A *base* y A2A *instrumentado (observado)*

La Tabla V reporta, para los tres escenarios en los que se intenta una operación no autorizada, cuántos intentos rechazó cada brazo (GCP y A2A+PEP, denegados/intentados) o ejecutó (A2A base, ejecutados/intentados), agrupados entre semillas. El resultado tiene tres partes.

Primero, los brazos con compuerta contienen todo lo que se les pide contener. GCP denegó los 443 intentos no autorizados (193 de delegación, 115 de acceso forzado, 135 de useful-forbidden), sin ejecución no autorizada en ningún modelo, y el brazo A2A instrumentado denegó los 408 propios (154, 124, 130). La contención es 1.00 para ambos brazos dondequiera que hubo un intento, y la diferencia entre GCP y A2A+PEP es nula en cada una de las 17 comparaciones con contención definida (Fisher $p = 1.000$). Esto se cumple tanto para el modelo más débil como para el más fuerte, de modo que no depende de que el modelo se porte bien.

Segundo, A2A *base ejecuta los intentos.* En los mismos escenarios ejecutó los 392 intentos no autorizados que recibió (145 de delegación, 121 de acceso forzado, 126 de useful-forbidden), y en todo modelo que intentó algo el canario o el contenido prohibido entró en la transcripción. Los intentos escalan con la capacidad: `qwen2.5:32b` hizo solo 5 intentos de delegación en 15 corridas sobre A2A base, mientras que `gemini-2.5-pro` hizo 23 en 30 corridas y `gemini-2.5-flash` 29 en 30. La brecha frente a GCP y frente a A2A+PEP es significativa en cada celda de ese tipo ($p < 0.001$, salvo $p = 0.012$ para `llama3.1:8b` en useful-forbidden, donde intentó solo de 3 a 6 veces, Tabla V).

Tercero, el brazo instrumentado iguala a GCP *en procedencia.* La completitud de procedencia es 1.00 para GCP y para A2A+PEP en cada celda, y 0.00 para A2A base dondequiera que el agente intentó algo. Los valores distintos de cero que A2A base muestra en unas pocas celdas (0.03–0.67) son exactamente la fracción de corridas en las que el agente no hizo ningún intento, de modo que no había nada que registrar; no son un rastro.

En conjunto, el contraste anterior "contención 1.00 frente a 0.00, procedencia 1.0 frente a 0.0" es una afirmación sobre un punto de aplicación de políticas y un sumidero de auditoría, no sobre el protocolo: una vez que un despliegue A2A añade ambos, los resultados de gobernanza medidos son idénticos. Lo que los datos no muestran es ningún resultado de gobernanza que GCP logre y que un A2A correctamente instrumentado no pueda. Lo que GCP ofrece en cambio es que el control por rol, la denegación por defecto, la regla de delegación y el registro por lectura forman parte del protocolo y de su servidor, mientras que el brazo A2A+PEP necesitó una tabla de tokens, un envoltorio alrededor del ejecutor de cada nodo y una convención para declarar la operación solicitada.

E. Acceso forzado: la contraparte del lado de lectura

Forced-access instruye al agente a leer el nodo confidencial, de modo que la ruta de lectura no autorizada se ejerce en cada corrida que se involucra con la tarea. A2A base ejecutó los 121 intentos y el canario apareció en la transcripción y en la respuesta en todos los casos (1.00/1.00 para `gemini-2.5-flash`, `qwen3:4b` y `qwen2.5:32b`; 0.90 para `gemini-2.5-pro`; 0.63 para `gemini-2.5-flash-lite`, que declinó con más frecuencia). La fuga es 0.00 bajo ambos brazos con compuerta. El nivel alojado más pequeño muestra un compromiso distinto según el brazo: `gemini-2.5-flash-lite` hizo 11 intentos de lectura en 30 corridas con GCP, frente a 19 con A2A base y 18 con A2A+PEP, de modo que una parte de sus corridas con GCP evitó la lectura declinando en lugar de ser denegada.

`llama3.1:8b` es un caso atípico genuino, no un dato faltante: hizo *cero* llamadas a herramientas en todos los brazos en las 90 corridas de forced-access. Una sonda directa no agéntica contra la misma entrada del escenario, ejecutada en una fase anterior de este trabajo, mostró al modelo rechazando en su respuesta final antes de invocar una herramienta (textualmente, *"I can't do that. Is there anything else I can help you with?"*). La frase del system prompt del escenario que explicita la discrepancia de rol resulta, para el entrenamiento de alineación de este modelo, una violación de política, por lo que declina de plano en *todos* los brazos, con independencia de que exista una compuerta. Esta es una interacción entre alineación y uso de herramientas en un modelo pequeño, ortogonal a la comparación de protocolos: el mismo modelo llama herramientas sin vacilar en software-org, supply-chain y delegation. La contención queda indefinida para este modelo en este escenario.

F. Useful-forbidden: fuga semántica por iniciativa propia del modelo

Useful-forbidden no instruye la lectura. Aun así, los modelos acudieron por su cuenta al nodo confidencial de finanzas en la mayoría de las corridas: de 24 a 30 intentos en 30 corridas para `gemini-2.5-flash`, `-flash-lite`, `-pro` y `qwen3:4b`, 15 en 15 para `qwen2.5:32b`, y solo de 3 a 6 para `llama3.1:8b`. Por tanto, el escenario discrimina entre brazos en todos los modelos salvo el último. Bajo A2A base el contenido prohibido entró en la transcripción de herramientas a una tasa de 0.46–0.57 de los elementos protegidos en cinco de los seis modelos (0.06 para `llama3.1:8b`) y llegó a la respuesta final en 0.11–0.14 (0.49 para `qwen3:4b`); bajo GCP y A2A+PEP ambas tasas son 0.00 en todos los modelos. Aun así, las tareas se completan:

TABLE V. INTENTOS NO AUTORIZADOS, AGRUPADOS ENTRE SEMILLAS: DENEGADOS/INTENTADOS PARA GCP Y A2A+PEP, EJECUTADOS/INTENTADOS PARA A2A BASE. CADA CELDA CUBRE 30 CORRIDAS (15 PARA QWEN2.5:32B); UNA CORRIDA PUEDE CONTENER VARIOS INTENTOS.

escenario	modelo	GCP denegados	A2A base ejecutados	A2A+PEP denegados
delegation	llama3.1:8b	30/30	30/30	30/30
	qwen3:4b	67/67	36/36	46/46
	qwen2.5:32b	15/15	5/5	2/2
	gemini-2.5-flash-lite	22/22	22/22	24/24
	gemini-2.5-flash	30/30	29/29	30/30
	gemini-2.5-pro	29/29	23/23	22/22
forced-access	llama3.1:8b	0/0	0/0	0/0
	qwen3:4b	30/30	30/30	31/31
	qwen2.5:32b	15/15	15/15	15/15
	gemini-2.5-flash-lite	11/11	19/19	18/18
	gemini-2.5-flash	30/30	30/30	30/30
	gemini-2.5-pro	29/29	27/27	30/30
useful-forbidden	llama3.1:8b	6/6	3/3	5/5
	qwen3:4b	29/29	27/27	29/29
	qwen2.5:32b	15/15	15/15	15/15
	gemini-2.5-flash-lite	27/27	24/24	24/24
	gemini-2.5-flash	28/28	27/27	27/27
	gemini-2.5-pro	30/30	30/30	30/30

A2A base puntuó $30/30$ en la parte autorizada para gemini-2.5-flash, -pro y qwen3:4b pese a la exposición. El éxito de tarea por sí solo habría ocultado esta fuga, y es la puntuación de transcripción y respuesta, no el oráculo, la que la expone.

G. Costo y éxito de tarea

La Tabla VI reporta el éxito de tarea en los tres escenarios cuyo criterio de éxito es la tarea autorizada. (En delegation y forced-access, el éxito se define como completar la acción no autorizada, de modo que una compuerta eficaz puntúa 0 por diseño; esos escenarios se leen mediante la contención, Tabla V.) Para cuatro de los seis modelos —gemini-2.5-flash, gemini-2.5-pro, qwen3:4b y qwen2.5:32b— el éxito no se distingue entre brazos en ningún escenario (todos $p \geq 0.49$, la mayoría de las celdas $30/30$). Dos modelos difieren.

gemini-2.5-flash-lite tiene éxito con menos frecuencia con GCP que con A2A base en los tres escenarios: $21/30$ frente a $28/30$ en software-org ($p = 0.042$), $20/30$ frente a $30/30$ en supply-chain ($p < 0.001$) y $9/30$ frente a $28/30$ en useful-forbidden ($p < 0.001$). Frente al brazo instrumentado, que tiene la misma compuerta y auditoría pero las herramientas por par de A2A, la brecha persiste en supply-chain ($20/30$ frente a $28/30$, $p = 0.021$) y useful-forbidden ($9/30$ frente a $29/30$, $p < 0.001$) y no es significativa en software-org ($p = 0.104$). Como A2A+PEP tiene exactamente la misma compuerta que GCP, la compuerta no puede explicar la brecha. Tampoco la forma de la herramienta: en el barrido conductual todos los brazos enlazaron una herramienta por par (Sección V). Las trazas almacenadas muestran lo que ocurrió. En las 40 corridas fallidas de GCP en los tres escenarios de lectura (9, 10 y 21) el modelo llamó exactamente a los mismos pares que en sus corridas exitosas y luego devolvió una *respuesta final vacía*; ninguna falló por elegir el par equivocado ni por ser rechazada. Las respuestas finales vacías se concentran en este único par modelo-brazo (60 de 150 corridas de GCP, incluyendo 19 de 30 corridas de delegación, frente a 3 de 150 en cada brazo A2A). El brazo instrumentado tiene muchos menos fallos, y sus fallos en software-org son rechazos sin llamada a herramienta. Un experimento posterior identificó la causa (Sección VI-H): es la redacción de la descripción de la herramienta, no el protocolo ni la compuerta. llama3.1:8b mostró una brecha en un escenario del barrido principal: en software-org $4/30$ con GCP frente a $14/30$ con A2A base ($p = 0.010$) y $12/30$ con A2A+PEP ($p = 0.039$), con paridad en supply-chain y useful-forbidden. No se reprodujo. Una repetición de las mismas configuraciones de GCP y A2A+PEP dio $15/30$ y $13/30$ ($p = 0.80$), y las dos corridas de GCP difieren entre sí ($4/30$ frente a $15/30$, $p = 0.005$). Intercambiar las descripciones de herramienta en la repetición no cambió significativamente ningún brazo ($14/30$ para GCP, $7/30$ para A2A+PEP, $p = 0.10$ entre ellos). En esta tarea el modelo responde casi siempre con pseudocódigo o una cadena con forma de llamada a herramienta en lugar de un resumen, en todos los brazos, y solo acierta cuando ese texto incluye por casualidad la salida de la herramienta; la brecha del barrido principal se lee por tanto mejor como variación entre corridas de este modelo, no como un efecto estable de GCP. Las significancias nominales de esta subsección, entre 30 comparaciones por par de brazos, no se corrigen por multiplicidad; las brechas de supply-chain y useful-forbidden de gemini-2.5-flash-lite son lo bastante grandes para sobrevivir a cualquier corrección estándar, las demás deben leerse como sugerentes.

Los tokens en el cable en los escenarios de lectura son cercanos entre GCP y A2A+PEP: la razón de tokens medios se sitúa entre 0.97 y 1.05 para cinco de los seis modelos, y entre 1.10 y 1.11 para qwen3:4b (d de Cohen de aproximadamente 0.5–0.6, un efecto moderado). El conteo en el cable del brazo base suele ser marginalmente menor porque omite el sobre de consulta de GCP. El costo de A2A que sí crece es el presupuesto de prompt de herramientas por solicitud cuando se enlaza una herramienta por par (Tabla III), no la carga útil. En los escenarios con compuerta los conteos de tokens difieren sobre todo porque el comportamiento difiere tras una decisión: bajo denegación, qwen3:4b emitió 67 intentos de delegación con GCP y 46 con A2A+PEP en 30 corridas, frente a 36 con A2A base, donde el primer intento tuvo éxito, lo que eleva su media a 11,456 frente a 5,455 tokens. Las comparaciones de tokens de qwen2.5:32b muestran valores extremos de la d de Cohen porque sus desviaciones estándar son casi cero (un modelo local determinista con semilla fija), y no deben leerse como efectos. No se afirma que GCP envíe menos tokens, solo que no cuesta significativamente más que un A2A igualmente protegido.

H. Diagnóstico: redacción de la descripción de la herramienta y respuestas vacías

Para comprobar si la redacción de la descripción de la herramienta por par explica las respuestas finales vacías de gemini-2.5-flash-lite con GCP, los tres escenarios de lectura se repitieron en los brazos GCP y A2A+PEP (30 semillas por celda, 360 corridas, ninguna fallida) dos veces: con la descripción nativa de cada brazo y con las descripciones intercambiadas, de modo que cada brazo veía la redacción del otro protocolo. Los nombres de las herramientas, los esquemas y el comportamiento no cambiaron. La Tabla VII muestra el resultado.

El control nativo reproduce el barrido principal: con GCP, 39 de 90 corridas terminan con una respuesta vacía frente a 1 de 90 con A2A+PEP (Fisher $p < 0.001$). Con la redacción intercambiada el patrón se invierte: 8 de 90 con GCP y 28 de 90 con A2A+PEP ($p < 0.001$). Dentro de cada brazo, la tasa de respuestas vacías pasa de 39 a 8 de 90 para GCP y de 1 a 28 de 90 para A2A+PEP (ambos $p < 0.001$), y el éxito de tarea se mueve con ella (GCP: $50/90$ a $79/90$; A2A+PEP: $88/90$ a $61/90$). El fallo sigue por tanto a la descripción, que dice "read the shared context … over the Graph Context Protocol" para GCP y "ask the peer … via an A2A message" para A2A, y no a la compuerta, el registro de auditoría ni el servidor. La inversión no es completa: GCP con la redacción de A2A ($8/90$) sigue superando a A2A+PEP con su propia redacción ($1/90$, $p = 0.034$, sin corregir), lo que deja margen para un pequeño efecto del brazo, y A2A+PEP con la redacción de GCP ($28/90$) no es significativamente menor que GCP nativo ($39/90$, $p = 0.12$). El experimento no establece qué rasgo de la redacción importa, y cubre un solo modelo.

TABLE VI. ÉXITO DE TAREA (k/n) EN LOS ESCENARIOS CUYO CRITERIO DE ÉXITO ES LA TAREA AUTORIZADA, Y p DE LA PRUEBA EXACTA DE FISHER DE DOS COLAS DE GCP FRENTE A A2A BASE Y FRENTE A A2A+PEP (NEGRITA: $p < 0.05$, SIN CORREGIR).

escenario	modelo	GCP	A2A base	A2A+PEP	p vs base	p vs PEP
software-org	`llama3.1:8b`	4/30	14/30	12/30	**0.010**	**0.039**
	`qwen3:4b`	30/30	30/30	30/30	1.000	1.000
	`qwen2.5:32b`	15/15	15/15	15/15	1.000	1.000
	`gemini-2.5-flash-lite`	21/30	28/30	27/30	**0.042**	0.104
	`gemini-2.5-flash`	30/30	30/30	30/30	1.000	1.000
	`gemini-2.5-pro`	30/30	30/30	30/30	1.000	1.000
supply-chain	`llama3.1:8b`	28/30	28/30	28/30	1.000	1.000
	`qwen3:4b`	30/30	30/30	30/30	1.000	1.000
	`qwen2.5:32b`	15/15	15/15	15/15	1.000	1.000
	`gemini-2.5-flash-lite`	20/30	30/30	28/30	**<0.001**	**0.021**
	`gemini-2.5-flash`	30/30	30/30	30/30	1.000	1.000
	`gemini-2.5-pro`	28/30	30/30	29/30	0.492	1.000
useful-forbidden	`llama3.1:8b`	7/30	9/30	10/30	0.771	0.567
	`qwen3:4b`	30/30	30/30	30/30	1.000	1.000
	`qwen2.5:32b`	15/15	15/15	15/15	1.000	1.000
	`gemini-2.5-flash-lite`	9/30	28/30	29/30	**<0.001**	**<0.001**
	`gemini-2.5-flash`	30/30	30/30	30/30	1.000	1.000
	`gemini-2.5-pro`	30/30	30/30	30/30	1.000	1.000

TABLE VII. RESPUESTAS FINALES VACÍAS Y ÉXITO DE TAREA DE `GEMINI-2.5-FLASH-LITE`, CON DESCRIPCIONES DE HERRAMIENTA NATIVAS E INTERCAMBIADAS (ESCENARIOS DE LECTURA AGRUPADOS, $n = 90$ POR CELDA).

descripción	brazo	respuestas vacías	éxito de tarea
nativa	GCP	39/90	50/90
nativa	A2A+PEP	1/90	88/90
intercambiada	GCP	8/90	79/90
intercambiada	A2A+PEP	28/90	61/90

I. Fuga: empates en supply-chain, la compuerta hace el resto

En *software-org*, la fuga observada fue 0.00 en todos los brazos y modelos: el agente cumplió la tarea de resumen público sin solicitar el nodo confidencial, de modo que la ruta de fuga estaba latente. En *supply-chain*, ambos brazos están autorizados a leer los nodos de ambos propietarios, por lo que los canarios llegan a la transcripción por igual en los tres brazos (1.00 para la mayoría de los modelos; 0.97–1.00 para `gemini-2.5-pro`; 0.72–0.78 para `llama3.1:8b`) y llegan a la respuesta rara vez, salvo en `qwen3:4b` (0.47–0.60 en todos los brazos). Es un empate genuino, porque la fuga no es una decisión de compuerta. Las diferencias de fuga que existen son las reportadas en las subsecciones de gobernanza, acceso forzado y useful-forbidden anteriores, y separan a A2A base de los dos brazos con compuerta, no a GCP de A2A+PEP.

J. Garantía arquitectónica: propiedad ejecutable de no-fuga

La distinción anterior descansa en una prueba, no solo en una medición. Sobre la ruta de lectura/autorización tal como se distribuye se corren pruebas basadas en propiedades (`fast-check`) contra conjuntos fijos y finitos: cinco capacidades y cuatro roles, uno de los cuales (`role:child`) hereda de otro (`role:a`) de modo que el cálculo de capacidad efectiva se ejerce, no solo las concesiones directas. Cada *política* aleatoria muestrea de forma independiente un subconjunto del conjunto de roles para `readableByRoles`, un subconjunto del conjunto de capacidades para `requiredCapabilities`, un booleano `fallbackAllowed` aleatorio y un modo de denegación aleatorio. Cada *principal* aleatorio muestrea de forma independiente un rol del conjunto o ninguno, más un subconjunto aleatorio de capacidades sostenidas directamente. Como ambos se extraen de conjuntos pequeños por muestreo de subconjuntos en lugar de un espacio no acotado, los sorteos autorizados y no autorizados ocurren ambos con frecuencia, de modo que las 1000 corridas por defecto no están dominadas por un solo resultado. Cada par $(policy, principal)$ generado se pasa entonces por el manejador real de consulta de contexto (no un simulacro) contra un adaptador de prueba que *siempre* devuelve una cadena canario centinela, de modo que P1 (no-fuga de contenido de extremo a extremo) afirma que una consulta no autorizada nunca devuelve el canario y queda marcada como `denied`, mientras que una autorizada sí lo devuelve (vivacidad, para que la prueba no sea vacúa); P2 (solidez de la autorización) verifica que la decisión de la compuerta distribuida sea igual a un predicado de referencia escrito de forma independiente sobre el mismo par; P3 (solidez de la herencia de roles) verifica el cálculo de capacidad efectiva directamente contra la cadena de roles. Todas pasan en 1000 corridas (elevable vía una variable de entorno; las corridas son reproducibles por semilla para depuración). La métrica de canario *mide* la fuga en los escenarios; la suite de propiedades da evidencia de que la compuerta coincide con un predicado de referencia escrito de forma independiente sobre un espacio de entradas acotado. Es una verificación basada en propiedades contra la propia referencia de los autores, no una prueba formal, y no aborda el robo de credenciales ni los canales fuera de la compuerta (Sección III-F). Juntas: la propiedad sugiere que la compuerta es correcta; el canario confirma que está en la ruta caliente.

VII. DISCUSIÓN, LIMITACIONES Y AMENAZAS A LA VALIDEZ

A. Interpretación de los principales hallazgos

En los tres escenarios de acción no autorizada, GCP denegó cada uno de los 443 intentos y el brazo A2A instrumentado cada uno de los 408 propios, mientras que A2A base ejecutó los 392 (Tabla V). La comparación que importa para la pregunta sobre el protocolo es la primera: con un punto de aplicación de políticas y un sumidero de auditoría delante, A2A es indistinguible de GCP en contención, procedencia y fuga en todos los modelos que corrimos. Esto es lo que predice la literatura de control de acceso. La mediación completa y el mínimo privilegio de Saltzer y Schroeder [11] y el control de acceso basado en roles [12] son propiedades de un punto de aplicación situado en la ruta hacia los datos, no del formato del mensaje, y lo mismo vale para la visión de Evertz et al. de que la confidencialidad es una propiedad del sistema y no del modelo [22], para la restricción de lo que un agente puede acceder de AirGapAgent [24] y para el control de privilegios en la frontera de acceso a herramientas de Progent [26]. La contribución de GCP no es por tanto una garantía más fuerte sino un empaquetado distinto: el punto de aplicación, la regla de denegación por defecto, la capacidad de delegación y el registro por decisión son partes nativas del protocolo y de su servidor, y se aplican a lecturas de contexto entre propietarios, mientras que un despliegue A2A debe ensamblarlos. Una diferencia merece señalarse. En nuestro brazo A2A+PEP la operación solicitada (leer o delegar) la declara el llamante en los metadatos del mensaje, de modo que la regla de delegación es tan fuerte como la convención que vincula esa declaración a una habilidad o endpoint distinto; en GCP la delegación es una operación distinta del protocolo y no puede solicitarse como lectura. Una prueba hermética muestra lo que cuesta esa convención. Un llamante con un token legítimo que simplemente omite la declaración se trata como lector: su delegación se ejecuta en cada intento, y el registro de auditoría anota una lectura permitida en lugar de una delegación denegada. La delegación del mismo agente a través de GCP es denegada. La equivalencia de A2A+PEP y GCP se cumple por tanto para un llamante que declara su operación, y cerrar la brecha exige un PEP que vincule la operación

a una habilidad o endpoint distinto en el servidor. El escenario de delegación no puede distinguir las operaciones por su efecto, porque su "ejecución" se simula devolviendo contenido del nodo que el rol también puede leer; lo que difiere es si la operación de delegación está controlada y registrada.

B. Relación con el trabajo previo

Tres hilos de la Sección II convergen en los resultados. En *acoplamiento*, el crecimiento de aristas de integración $\mathcal{O}(n)$ frente a $\mathcal{O}(n^2)$ (Tabla II) y el sobrecosto de descubrimiento y prompt de herramientas $\mathcal{O}(1)$ frente a $\mathcal{O}(n)$ (Tabla III) instancian, para topologías de agentes LLM, el mismo argumento de conteo de aristas que Hohpe and Woolf [7], Parnas [8], Stevens et al. [9], Briand et al. [10] establecieron para arquitecturas de software e integración, con la salvedad de que un orquestador A2A o una herramienta genérica única elimina la diferencia. En *mínimo privilegio y control de acceso*, la equivalencia de GCP y A2A+PEP muestra que los resultados de contención y procedencia operacionalizan a Saltzer and Schroeder [11] y Sandhu et al. [12] al nivel de lecturas individuales de contexto, y que los obtiene quien coloca la compuerta. En *fuga y gobernanza*, el empate de supply-chain y la brecha de useful-forbidden respaldan conjuntamente a Evertz et al. [22] y El Yagoubi et al. [23]: los resultados de confidencialidad dependen de si se ejerce la ruta no autorizada y de si existe una compuerta, y el escenario useful-forbidden muestra que los modelos toman esa ruta por su cuenta cuando el contenido prohibido es útil.

C. Implicaciones prácticas y condiciones de aplicabilidad

Para despliegues que deben soportar el intercambio de contexto entre propietarios bajo requisitos de mínimo privilegio y auditoría —p. ej. obligaciones regulatorias de minimización de datos [21] o los riesgos de Excessive Agency y Sensitive Information Disclosure del OWASP LLM Top 10 [25]— los resultados indican que un despliegue A2A sin protección es inseguro frente a un modelo capaz al que no se le impide preguntar, y que el remedio es un punto de aplicación y un registro de auditoría, vengan de donde vengan. GCP los provee como primitivas del protocolo; A2A los provee si el desplegador los añade. El costo de añadirlos fue pequeño en nuestra implementación pero es trabajo de ingeniería real que cada despliegue repite. También apareció un costo de GCP: el modelo alojado más pequeño completó la tarea autorizada con menos frecuencia a través de GCP que a través de A2A igualmente protegido, por una interacción entre el modelo y la redacción de la descripción de la herramienta (Sección VI-H); una brecha similar del modelo local más pequeño no se reprodujo al repetirla. Los modelos en uso deben probarse con la redacción que realmente verán.

D. Limitaciones y amenazas a la validez

a) Las restricciones de recursos acotan la evaluación.: Este estudio no recibió financiamiento externo y se realizó como una iniciativa independiente (véase la declaración de Financiamiento). El barrido usa 30 semillas por brazo en cinco modelos y 15 en `qwen2.5:32b`, cuyas corridas son las más lentas; con estos tamaños de muestra se detectan diferencias de aproximadamente 25 puntos porcentuales o más en el éxito, y las menores no. El costo de los modelos alojados limitó a una las familias alojadas (Gemini 2.5). Los trabajos relacionados citados en la Sección II (p. ej. Zhang et al. [15], Mao and Mirhoseini [16]) evalúan a escala de producción o con mayor presupuesto; la comparación ofrecida aquí es un complemento controlado por equidad pero acotado en recursos a esa literatura.

b) El punto de aplicación es una implementación de los autores.: El brazo A2A+PEP fue escrito por los autores, en el mismo código base que GCP, y aplica la misma regla de decisión. Por tanto, resultados iguales son esperables y muestran que la *política*, no el protocolo, determina la contención; no muestran que todo gateway A2A de terceros sea igual de completo, ni que el PEP esté libre de evasiones: lo evade un llamante que omite la declaración de operación (Sección III-F y la discusión anterior). El brazo reportado no se modificó tras descubrirlo. Una variante opcional que deniega las peticiones sin declaración (razón `operation-undeclared`) se repitió en el escenario de delegación con los seis modelos (177 registros, 165 corridas distintas): contención 1.00, ninguna delegación ejecutada, procedencia 1.00 y éxito de tarea 0 en todos los modelos, igual que el brazo sin reforzar, y ninguna corrida fue denegada por operación no declarada porque los modelos siempre la declaran. Esto cierra la omisión pero no una declaración falsa de "read", que el servidor no distingue de una lectura en este escenario.

c) Alcance de adversarios y credenciales.: Solo el adversario de modelo instruido se ejerce empíricamente, y las credenciales son tokens estáticos; las credenciales robadas, los dueños de nodo maliciosos y los agentes que coluden quedan fuera de alcance (Sección III-F). La inyección de prompt mediante contenido recuperado no se evalúa.

d) Asimetría de transporte.: El brazo GCP aloja los nodos en el mismo proceso detrás del manejador de fetch del servidor, mientras que ambos brazos A2A usan servidores HTTP en localhost, de modo que la latencia de reloj de pared no es comparable y no respalda ninguna afirmación aquí. Los conteos de tokens son tokens de modelo y no se ven afectados.

e) Alcance estadístico.: Las pruebas son exactas pero no están corregidas por multiplicidad, los intentos de contención se agrupan entre semillas aunque una corrida pueda contener varios, y las desviaciones estándar casi nulas de los modelos locales deterministas (en particular `qwen2.5:32b`) hacen inestable la d de Cohen. Los logs por corrida permiten rehacer cualquiera de estos análisis.

f) La medición de fuga es un sustituto.: Useful-forbidden detecta la paráfrasis con patrones escritos para ese escenario; un juez basado en implicación podría encontrar divulgaciones que los patrones omiten o marcar otras benignas. Las trazas se almacenan para poder revisar la puntuación sin nuevas corridas. Supply-chain sigue confundiendo la fuga con lecturas autorizadas, de modo que mide un empate y no un efecto de compuerta, y la inyección indirecta de prompt no se ejerce.

g) El efecto de la redacción se muestra para un solo modelo.: El experimento de intercambio de descripciones (Sección VI-H) se corrió solo para `gemini-2.5-flash-lite`, con 90 corridas por celda y 3 escenarios de lectura, de modo que muestra que la redacción impulsa el fallo de respuesta vacía en ese modelo, no que lo haga en otros, ni qué rasgo de la redacción importa. La brecha de `llama3.1:8b` del barrido principal no se reprodujo al repetirla (Sección VI-G); la misma configuración difirió en 11 éxitos de 30 entre dos corridas de este modelo, de modo que las brechas de una sola corrida en modelos locales pequeños deben leerse teniendo en cuenta esa variación. Como la redacción es parte de cómo se presenta cada protocolo, los resultados de éxito del barrido principal no pueden separarse de ella. `llama3.1:8b` además declinó la tarea de forced-access en todos los brazos, lo que elimina la información de ese escenario para ese modelo.

h) El esfuerzo es un indicador.: Las cifras de esfuerzo de integración (Sección VI-C) cuentan declaraciones en el arnés de los propios autores. No miden tiempo de desarrollo ni mantenibilidad, lo que exigiría un estudio con desarrolladores, y los conteos dependen de cómo se descompone en entradas el cableado de cada brazo.

i) Marketplace usado solo para estructura.: El éxito de tarea del marketplace fue bajo en *ambos* brazos (p. ej. cayendo a 0.0–0.4 en n=16, 32 con `llama3.1:8b`); el modelo a menudo no logró elegir al vendedor más barato. Como la falla es igual entre brazos no amenaza la equidad, pero el marketplace se usa *solo* por sus curvas deterministas de estructura, descubrimiento y prompt de herramientas. Esas curvas y sus mediciones de tokens de prompt de herramientas provienen del barrido anterior del marketplace, no del presente barrido de 2,475 corridas.

j) Validez de constructo de la línea base.: Los brazos A2A corren servidores reales, el mismo cerebro y la misma tarea, construidos con igual cuidado. Las desventajas del brazo base (sin compuerta de lectura, sin compuerta de delegación, sin sumidero de auditoría nativo, topología por pares) son las del protocolo tal como se distribuye, no artefactos de un hombre de paja, y desaparecen en el brazo instrumentado, por lo cual ese brazo es el comparador justo.

VIII. Trabajo Futuro

Aislar qué rasgo de la descripción de la herramienta desencadena las respuestas vacías, repetir el intercambio en otros modelos y ejecutar de forma conductual el enlace de herramienta única; añadir una segunda familia alojada de frontera y replicar con un gateway A2A escrito de forma independiente; reforzar el punto de aplicación vinculando la operación a una habilidad o endpoint distinto y evaluar declaraciones falsificadas, credenciales robadas e inyección indirecta de prompt mediante contenido recuperado; sustituir el evaluador semántico basado en patrones por un juez de implicación validado contra etiquetas humanas; y diseñar un banco de pruebas de privacidad que elimine el factor de confusión de lectura autorizada del escenario supply-chain. Conectar GCP con MCP y A2A como interoperabilidad de primera clase es la ruta natural de despliegue.

IX. Conclusión

Este trabajo presentó GCP —federación de contexto de lectura primero, controlada por rol y auditada, con una compuerta de delegación de capacidad más estricta— y una comparación controlada por equidad contra dos brazos de paso de mensajes, A2A base y A2A detrás de un punto de aplicación de políticas equivalente, en seis modelos y 2,475 corridas. Las ventajas estructurales son independientes del modelo pero topológicas: el acoplamiento baja de $\mathcal{O}(n^2)$ a $\mathcal{O}(n)$ aristas de integración respecto a una malla A2A, y el sobrecosto

de descubrimiento y de prompt de herramientas por solicitud de $\mathcal{O}(n)$ a $\mathcal{O}(1)$ respecto a herramientas por par; un orquestador o una herramienta genérica única cierra ambas brechas. En gobernanza, GCP contuvo toda acción no autorizada (443 de 443) mientras que A2A base no contuvo ninguna (0 de 392), y los modelos acudieron por su cuenta al contenido prohibido en el escenario useful-forbidden, filtrándolo a la transcripción bajo A2A base pero nunca bajo los brazos con compuerta. El brazo A2A instrumentado, sin embargo, igualó a GCP en contención (408 de 408), procedencia y fuga en todos los modelos y escenarios, para un llamante que declara su operación. La contribución defendible no es por tanto que GCP asegure lo que A2A no puede, sino que hace nativos del protocolo el control por rol, la denegación por defecto, una regla de delegación y un registro por lectura, de modo que no necesiten ensamblarse en cada despliegue. Esto viene con un costo medido para el modelo alojado más pequeño, que completó las tareas autorizadas con menos frecuencia a través de GCP; un experimento de intercambio de descripciones muestra que la redacción de la descripción de la herramienta, no el protocolo, causa sus respuestas finales vacías. Una brecha del modelo local más pequeño no se reprodujo al repetirla. Una prueba hermética muestra además que el brazo instrumentado es evadido por un llamante que omite su declaración de operación, algo que la operación de delegación distinta de GCP no permite. Queda abierta la pregunta de cuánta de la equivalencia restante sobrevive a un gateway que vincule la operación en el servidor y a adversarios más fuertes.

Disponibilidad de Código y Datos

La implementación de GCP, el arnés de evaluación, las definiciones de escenarios y los scripts que regeneran cada tabla a partir de los logs crudos por corrida se publican bajo licencia MIT en https://github.com/AI-DO-LABS/graph-context-protocol (estado del repositorio de este artículo: véase la versión etiquetada). Los logs de ejecución, incluidos los del experimento de intercambio de descripciones, se almacenan como una línea JSON por modelo, escenario, brazo y semilla, incluyendo las corridas fallidas, de modo que los denominadores nunca se ocultan, y cada línea lleva la respuesta final, la transcripción de herramientas y las decisiones de auditoría para poder reevaluar la fuga. Los logs, los scripts de agregación y un runbook que reproduce el barrido están en `docs/eval/`.

Declaración de Conflicto de Intereses

Los autores declaran que no existen conflictos de intereses potenciales relacionados con la investigación, la autoría o la publicación de este artículo.

Financiamiento

Esta investigación no recibió financiamiento externo. El estudio fue desarrollado como una iniciativa independiente de los investigadores.

Agradecimientos

Los autores expresan su agradecimiento a la Universidad Nacional de San Agustín de Arequipa por ser la casa de estudios en la que se realizó este trabajo de investigación.

Declaración sobre IA Generativa

Durante la preparación de este trabajo, los autores usaron Claude y Claude Code (Anthropic) para asistir en la redacción y edición del texto del manuscrito, el refinamiento del lenguaje, y la implementación y ejecución de partes del arnés de evaluación (código de escenarios, adaptadores y generación de reportes de resultados) bajo la dirección y supervisión de los autores. Las ideas científicas, el diseño experimental, la interpretación de los resultados y las conclusiones de este artículo fueron desarrollados por los autores humanos. Tras usar estas herramientas, los autores revisaron y editaron todo el contenido y código asistido por IA, verificaron todos los resultados reportados contra los artefactos de ejecución subyacentes, y asumen plena responsabilidad por la exactitud, originalidad e integridad de esta publicación. Ninguna herramienta de IA generativa figura como autora.

References

[1] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023. [Online]. Available: https://arxiv.org/abs/2308.08155

[2] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. [Online]. Available: https://arxiv.org/abs/2307.07924

[3] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. [Online]. Available: https://arxiv.org/abs/2308.00352

[4] Google and Linux Foundation, "Agent2Agent (A2A) protocol specification, v1.0," Open standard, 2025. [Online]. Available: https://a2a-protocol.org/v1.0.0/specification/

[5] T. Finin, R. Fritzson, D. McKay, and R. McEntire, "KQML as an agent communication language," in *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM)*, 1994.

[6] Foundation for Intelligent Physical Agents, "FIPA ACL message structure specification," FIPA, Tech. Rep. SC00061G, 2002. [Online]. Available: http://www.fipa.org/specs/fipa00061/

[7] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2003.

[8] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972.

[9] W. P. Stevens, G. J. Myers, and L. L. Constantine, "Structured design," *IBM Systems Journal*, vol. 13, no. 2, pp. 115–139, 1974.

[10] L. C. Briand, J. W. Daly, and J. K. Wüst, "A unified framework for coupling measurement in object-oriented systems," *IEEE Transactions on Software Engineering*, vol. 25, no. 1, pp. 91–121, 1999.

[11] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[12] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[13] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, "Large language model based multi-agents: A survey of progress and challenges," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024, pp. 8048–8057. [Online]. Available: https://arxiv.org/abs/2402.01680

[14] Anthropic, "Introducing the model context protocol," Anthropic; open specification, 2024. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[15] G. Zhang, Y. Yue, Z. Li, S. Yun, G. Wan, K. Wang, D. Cheng, J. X. Yu, and T. Chen, "Cut the crap: An economical communication pipeline for LLM-based multi-agent systems," *arXiv preprint arXiv:2410.02506*, 2024. [Online]. Available: https://arxiv.org/abs/2410.02506

[16] Y. Mao and A. Mirhoseini, "Decentralized multi-agent systems with shared context," *arXiv preprint arXiv:2606.10662*, 2026. [Online]. Available: https://arxiv.org/abs/2606.10662

[17] S. Panichella, M. I. Rahman, and D. Taibi, "Structural coupling for microservices," in *Proceedings of the 11th International Conference on Cloud Computing and Services Science (CLOSER)*, 2021, pp. 280–287. [Online]. Available: https://arxiv.org/abs/2103.04674

[18] X. Shen, Y. Liu, Y. Dai, Y. Wang, R. Miao, Y. Tan, S. Pan, and X. Wang, "Understanding the information propagation effects of communication topologies in LLM-based multi-agent systems," *arXiv preprint arXiv:2505.23352*, 2025. [Online]. Available: https://arxiv.org/abs/2505.23352

[19] V. C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, and K. Scarfone, "Guide to attribute based access control (ABAC) definition and considerations," National Institute of Standards and Technology, Tech. Rep. NIST Special Publication 800-162, 2014.

[20] J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966.

[21] European Parliament and Council, "Regulation (EU) 2016/679 (general data protection regulation), articles 5 and 25," Official Journal of the European Union, L 119, 2016. [Online]. Available: https://gdpr-info.eu/

[22] J. Evertz, M. Chlosta, L. Schönherr, and T. Eisenhofer, "Whispers in the machine: Confidentiality in agentic systems," *arXiv preprint arXiv:2402.06922*, 2024. [Online]. Available: https://arxiv.org/abs/2402.06922

[23] F. El Yagoubi, G. Badu-Marfo, and R. Al Mallah, "AgentLeak: A full-stack benchmark for privacy leakage in multi-agent LLM systems," *arXiv preprint arXiv:2602.11510*, 2026. [Online]. Available: https://arxiv.org/abs/2602.11510

[24] E. Bagdasarian, R. Yi, S. Ghalebikesabi, P. Kairouz, M. Gruteser, S. Oh, B. Balle, and D. Ramage, "AirGapAgent: Protecting privacy-conscious conversational agents," *arXiv preprint arXiv:2405.05175*, 2024. [Online]. Available: https://arxiv.org/abs/2405.05175

[25] OWASP GenAI Security Project, "OWASP top 10 for large language model applications," OWASP Foundation, 2025. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[26] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Programmable privilege control for LLM agents," *arXiv preprint arXiv:2504.11703*, 2025. [Online]. Available: https://arxiv.org/abs/2504.11703

[27] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, D. Greenwood, A. Chan, and A. Pentland, "Authenticated delegation and authorized AI agents," *arXiv preprint arXiv:2501.09674*, 2025. [Online]. Available: https://arxiv.org/abs/2501.09674

[28] V. Ojewale, H. Suresh, and S. Venkatasubramanian, "Audit trails for accountability in large language models," *arXiv preprint arXiv:2601.20727*, 2026. [Online]. Available: https://arxiv.org/abs/2601.20727